

Concept Primer for the Arithmetic and Logic Challenges

Background to read before we start

Read this before the exercise. It explains, in plain language and with a small worked example each, the ideas the challenges are built on, one section per idea. It does not tell you how to solve any particular puzzle. The point is that when you sit down to the challenges, none of the underlying concepts are new. Each section ends with a line on why the idea matters outside the classroom, because in every case it does.

How to use this. Skim it once now, then keep it open during the exercise as a reference. If a challenge makes you think "wait, how are numbers even stored here," the answer is probably a paragraph in this handout.

1. Number bases, bytes, and hexadecimal

A computer stores everything as bits, and bits are grouped into bytes of eight. Binary is base 2, so each bit is a power of two. Hexadecimal is base 16, and it exists mostly for human convenience: one hex digit stands for exactly four bits, so two hex digits describe one byte exactly. That tidy two-digits-per-byte fit is why raw data is almost always shown in hex. Text is just bytes too: a character set such as ASCII assigns each character a number, so the same bytes can be read as numbers or as letters depending on how you choose to interpret them.

Example. The letter H has code 72. That one byte can be written three ways that all mean the same eight bits, and two bytes together can spell a word:

binary	01001000	hex	48 69	
hex	48	text	H i	# 48 -> 'H', 69 -> 'i'
decimal	72	so	48 69 reads	"Hi"

Why it matters. File formats, memory contents, and network traffic are all just bytes. Moving fluently between binary, hex, and text is the baseline skill for looking at any of them.

2. Signed integers and two's complement

A fixed-width integer is just a pattern of bits, and the same pattern can mean different things depending on whether you treat it as signed or unsigned. The common scheme for signed numbers is two's complement, in which the top bit carries negative weight. Under it, the same bits that read as a large positive number when unsigned can read as a negative number when signed. The practical lesson is that a program decides, in its choice of type, whether a value is allowed to be negative at all, and that subtracting a negative is the same as adding.

Example. Read the same eight ones two different ways, and watch a subtraction turn into an addition:

```

11111111 as unsigned = 255
11111111 as signed   = -1      # top bit carries negative weight

balance - (-50) = balance + 50 # subtracting a negative adds

```

Why it matters. Choosing the type of a variable is a security decision, not only a storage one. A great many bugs begin with a value being negative when the programmer assumed it could not be.

3. Integer overflow and wraparound

Fixed-width integers cannot grow without bound. When a value climbs past the largest number its bits can hold, it does not keep going; it wraps around. In a signed type, pushing just past the maximum rolls over to the most negative value, the way a car odometer rolls from all nines back to zeros. This is not a rare curiosity: the arcade game Pac-Man has a famous broken level 256 caused by an eight-bit level counter overflowing, and in 2015 the aviation authority ordered operators of the Boeing 787 to power the aircraft off periodically, because a signed 32-bit counter would overflow after about 248 days of continuous operation and put certain systems into a failure state.

Example. In an eight-bit signed integer the largest value is 127. Adding one more does not give 128; it wraps to the most negative value:

```

 01111111 = 127
+ 00000001 =  1
-----
10000000  = -128    # one past the top lands at the very bottom

```

Why it matters. Counters, sizes, lengths, and prices all live in fixed-width integers. Whenever arithmetic can push one past its limit, the result may be the opposite of what everyone expected.

4. XOR and why it can be undone

XOR, the exclusive-or operation, outputs 1 when its two inputs differ and 0 when they are the same.

Table 1: The XOR truth table.

Input A	Input B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

The property that makes XOR central to cryptography is that applying the same value twice returns you to the start. This is the part that is easy to find confusing, so here it is spelled all the way out.

Example, one bit at a time. XORing a bit with 1 flips it. Do it a second time and it flips back:

```

0 XOR 1 = 1      # flipped
1 XOR 1 = 0      # flipped back to where it started

```

Example, a whole letter. Encrypt H by XORing it, column by column, with a key byte:

```
01001000    # H = 72
00101010    # key
-----    XOR
01100010    # cipher = 98, which happens to be 'b'
```

Now decrypt by XORing the cipher with the very same key:

```
01100010    # cipher
00101010    # the same key again
-----    XOR
01001000    # back to H = 72
```

It undoes itself because, in every column, the key bit was XORed in twice. A bit XORed with the same value twice returns to itself, since a value XORed with itself is 0, and anything XORed with 0 is left unchanged. Written out:

```
(message XOR key) XOR key
= message XOR (key XOR key)
= message XOR 0
= message                # the key cancels itself out
```

Encryption and decryption are the same operation. That is powerful, but it also means a weak key gives no protection: a one-byte key has only 256 possibilities, and trying all of them takes no time at all.

Why it matters. XOR appears in nearly every cipher. Understanding both its reversibility and how little a short key protects is the start of thinking about what makes encryption strong or weak.

5. Reusing a key

A one-time pad, in which a truly random key as long as the message is used exactly once, is unbreakable. The danger is in the word once. If the same key is used to encrypt two different messages, then combining the two ciphertexts with XOR cancels the key out entirely and leaves the two messages combined with each other. The secret key has removed itself from the problem.

Example. Encrypt two letters with the same key, then XOR the two ciphertexts together:

```
A = 01000001    with key 00001111 -> c1 = 01001110
C = 01000011    with key 00001111 -> c2 = 01001100

c1 XOR c2 = 00000010
A XOR C   = 00000010    # identical: the shared key is gone
```

The two ciphertexts XORed together equal the two plaintexts XORed together, with no key anywhere in the result.

Why it matters. Key reuse is one of the most common ways otherwise sound encryption is broken in practice. The rule that a keystream must never be reused comes directly from this.

6. Boolean algebra, gates, and simplification

Digital logic is built from a small set of operations: AND, OR, NOT, and XOR. Each has a truth table, and any condition a program checks can be written as a combination of them. A password or key check, for example, is really a Boolean function of the bits of the input. Your chapter gives the tools to write such a function as a sum of product terms, simplify it with the basic identities, and group it on a Karnaugh map to find its simplest form.

Example. The function (A AND B) OR (A AND NOT B) looks like it needs both inputs, but its truth table tells a simpler story:

Table 2: The whole expression is 1 exactly when A is 1.

A	B	(A AND B) OR (A AND NOT B)
0	0	0
0	1	0
1	0	1
1	1	1

The output is 1 whenever A is 1, regardless of B, so the entire expression simplifies to just A. Reducing a check like this is how you learn what an input actually has to satisfy.

A word on Karnaugh maps. Karnaugh maps take practice, and they can be confusing the first few times you use one. Do not worry if they do not make sense right away. You will not need one to finish any of the challenges, so treat a Karnaugh map as something to explore, not something to master before we start.

Why it matters. Every branch and check in software is logic. Being able to read it, write its truth table, and reduce it tells you exactly what an input would have to look like to take a particular path.

7. Floating point, approximation, and special values

Floating point is how computers store numbers with fractional parts, and the key fact about it is that it is an approximation. Many ordinary decimal values, including something as simple as one tenth, cannot be represented exactly in binary, so a tiny error is baked in and can accumulate. In 1991 a Patriot missile battery tracked time in tenths of a second using this format; the small rounding error built up over about a hundred hours of running, the internal clock drifted by roughly a third of a second, and an incoming missile was missed. The standard also defines special values that are not ordinary numbers, including the infinities and a value called NaN, meaning Not a Number, and these follow their own surprising rules when compared.

Example. A sum you would expect to be exact is not, and NaN refuses every comparison you make with it:

```
0.1 + 0.2 is 0.30000000000000004, not 0.3 # so it is not "equal to 0.3"

NaN < 5      is false
NaN > 5      is false
NaN = NaN    is false      # not even equal to itself
```

Why it matters. Any program doing measurement, money, timing, or physics relies on floating point. Its approximate nature and its special values are a frequent source of bugs that only appear at the edges.

8. Flip-flops, shift registers, and the LFSR

A flip-flop is the simplest memory element: a circuit that stores one bit. Line several up and connect each to its neighbor, and you have a shift register, where on every clock tick each stored bit moves one position along. Now add one wire. Instead of letting the bit at the end fall away, compute a new bit by taking the XOR of a few chosen positions, called taps, and feed it back into the other end. That combination, a shift register with an XOR feedback tap, is a linear feedback shift register, or LFSR. Clock it and it produces a long stream of bits that looks random even though it is completely deterministic: the same starting value, called the seed, always produces the same stream.

Example. Take a tiny four-bit register whose feedback bit is the XOR of its two rightmost bits, started at 1000. Each step shifts right, drops the feedback bit in on the left, and puts that feedback bit out as the output:

```
state 1000  -> output 0
state 0100  -> output 0
state 0010  -> output 1
state 1001  -> output 1
state 1100  -> output 0
state 0110  -> output 1      # output so far: 0 0 1 1 0 1
```

The output looks patternless, yet nothing random happened, and the same seed will reproduce it exactly. XOR that stream against a message and you have a simple stream cipher built from nothing but flip-flops and one XOR gate.

Why it matters. The LFSR is the bridge from sequential logic to cryptography. It shows how a handful of gates can generate a keystream, and it is the starting point for understanding real stream ciphers.

9. How much a generator like that really hides

Here is the idea to carry into the hardest challenge. A generator built from a register is only as unpredictable as that register is wide. All of its future output is determined by the bits currently stored in it, so if an attacker ever learns the contents of the register, every bit it will ever produce is known.

Example. The four-bit register above holds only four bits of state, so once you have seen four of its output bits you have effectively seen the whole register, and every later bit is already fixed. A register with sixteen bits of state is no different in kind; it simply takes sixteen observed bits instead of four before the same thing is true.

Why it matters. This is why serious ciphers use very large internal states and mix them in ways that are not purely linear. A short, simple generator is fine for teaching and useless for protecting anything real, and knowing the difference is the whole lesson.